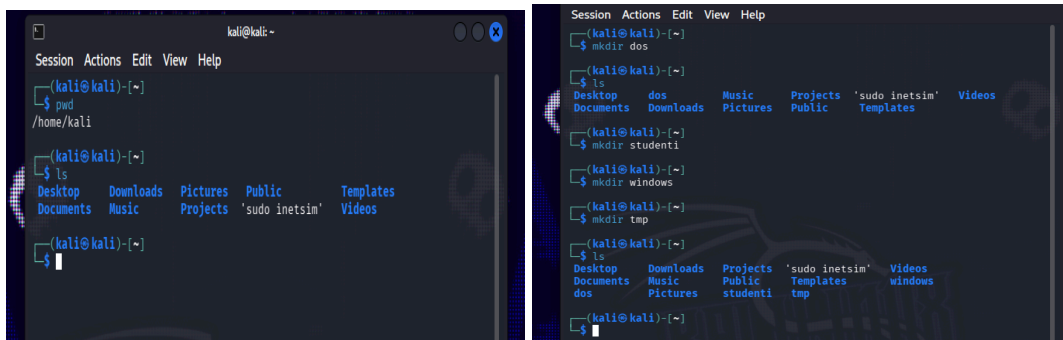


Report esercizio Shell Linux

Eseguita esercitazione sui comandi della shell di linux e la gestione delle risorse e dei dati tramite questi comandi e dei processi.

Come prima cosa si avvia la macchina e si fa il log-in.

Come secondo passo si inizia a creare le cartelle e sottocartelle usando il terminale e il relativo comando:

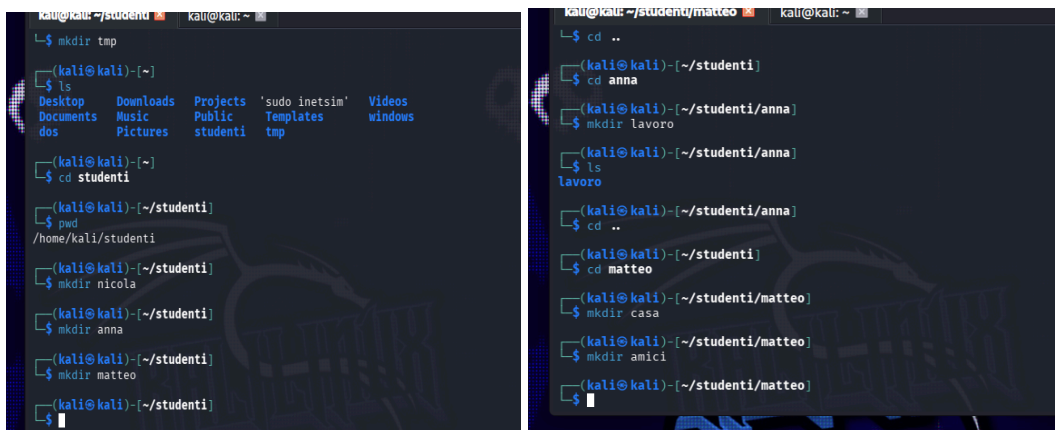


```
kali@kali: ~  
$ pwd  
/home/kali  
$ ls  
Desktop  Downloads  Pictures  Public  Projects  'sudo inetsim'  Templates  Videos  
$  
$ mkdir dos  
$ mkdir studenti  
$ mkdir windows  
$ mkdir tmp  
$ ls  
Desktop  Downloads  Projects  'sudo inetsim'  Videos  
Documents Music      Public    Templates      tmp      windows  
$
```

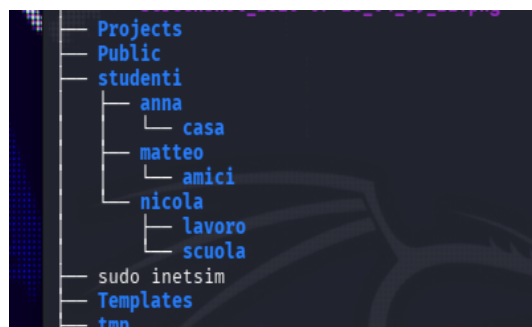
Per creare le cartelle è sufficiente utilizzare il comando **mkdir [nome della cartella]** e premere invio. Per visualizzare le cartelle in elenco si utilizza il comando **ls**, mentre per visualizzare in quale cartella ci si trova il comando **pwd**.

Per creare le sottocartelle è necessario essere dentro la cartella di riferimento, per fare ciò utilizziamo il comando **cd [nome della cartella]**.

Per tornare indietro tra le cartelle il comando **cd ..**, per tornare alla home del terminale **cd** semplicemente o **cd *tilde***, che si trova premendo alt gr 1.



```
kali@kali: ~/studenti  
$ mkdir tmp  
$ ls  
Desktop  Downloads  Projects  'sudo inetsim'  Videos  
Documents Music      Public    Templates      windows  
dos      Pictures  studenti  tmp  
$ cd studenti  
$ pwd  
/home/kali/studenti  
$ mkdir nicola  
$ mkdir anna  
$ mkdir matteo  
$  
kali@kali: ~/studenti/matteo  
$ cd ..  
$ mkdir lavoro  
$ ls  
lavoro  
$ cd ..  
$ cd matteo  
$ mkdir casa  
$ mkdir amici  
$
```



Infine una volta create le cartelle e visualizzare l'intero sistema/home/kali il comando **tree**. (Nel secondo screen delle sottocartelle le ho invertite e ripristinate poi col comando **mv [nome cartelle/sottocartelle]**.

Come seconda richiesta abbiamo di eseguire i percorsi relativi ed assoluti da una cartella a un'altra; cio si puo fare col comando **cd ../../[nome cartella/sottocartella]** per quello relativo, **cd /home/kali/[nome cartella/sottocartella]** per quello assoluto.

```
(kali㉿kali)-[~]
$ cd studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ cd ../../anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$
```

```
(kali㉿kali)-[~]
$ cd /home/kali/studenti/anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$
```

La seconda parte dell'esercizio prevede diversi punti da svolgere:

a) copiare il documento compito.doc, creato col comando **touch**, dalla cartella lavoro a casa col comando **cp**.

```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ touch studenti/nicola/scuola/compito.doc

(kali㉿kali)-[~]
$ cp studenti/nicola/scuola/compito.doc studenti/anna/casa

(kali㉿kali)-[~]
$ cd /home/kali/studenti/anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$ ls
compito.doc

(kali㉿kali)-[~/studenti/anna/casa]
$
```

b) spostare il documento relazione.doc dalla cartella lavoro a casa col comando **mv**.

```
(kali㉿kali)-[~]
$ mv studenti/nicola/scuola/relazione.doc studenti/anna/casa

(kali㉿kali)-[~]
$ ls studenti/nicola/scuola
compito.doc

(kali㉿kali)-[~]
$ ls studenti/anna/casa
compito.doc relazione.doc

(kali㉿kali)-[~]
$
```

c) cancellare la cartella tmp col comando **rm -r**

```
(kali㉿kali)-[~]
$ rm -r tmp

(kali㉿kali)-[~]
$ ls
Desktop      Downloads    Projects    'sudo inetsim'  windows
Documents    Music        Public      Templates
dos          Pictures     studenti    Videos
```

d) creare il file pippo.txt nella cartella lavoro

```
(kali㉿kali)-[~]
$ touch studenti/nicola/lavoro/pippo.txt

(kali㉿kali)-[~]
$ ls studenti/nicola/lavoro
pippo.txt

(kali㉿kali)-[~]
$
```

e) cambiare gli attributi del file pippo.txt e renderlo leggibile e scrivibile solo per il proprietario col comando **chmod 644**, 644 indica i permessi attribuiti al proprietario, al gruppo e agli altri

```
(kali㉿kali)-[~]
$ chmod 644 studenti/nicola/lavoro/pippo.txt

(kali㉿kali)-[~]
$ ls -l studenti/nicola/lavoro
total 0
-rw-r--r-- 1 kali kali 0 Jul 28 06:19 pippo.txt

(kali㉿kali)-[~]
$
```

f) nascondere il contenuto della cartella anna col comando **mv [nome cartella] .[nome cartella]**, col comando **ls -a** si vede il contenuto nascosto

```
(kali㉿kali)-[~]
$ cd studenti/anna

(kali㉿kali)-[~/studenti/anna]
$ mv casa .casa

(kali㉿kali)-[~/studenti/anna]
$ ls

(kali㉿kali)-[~/studenti/anna]
$ ls -a
.  ..  .casa

(kali㉿kali)-[~/studenti/anna]
$
```

g) spostarsi nella cartella lavoro e visualizzare il contenuto del file pippo.txt. Il

comando per visualizzare il contenuto di una cartella è **cat** ma dato che la cartella creata per l'esercizio era vuota si può creare del testo col comando **echo** "testo">[nome del file]

```
(kali㉿kali)-[~]
└─$ cd studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
└─$ pwd
/home/kali/studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
└─$ cat pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
└─$ echo "File Pippo.txt">pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
└─$ cat pippo.txt
File Pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
└─$
```

h) rimuovere la cartella amici col comando **rm -r**

```
(kali㉿kali)-[~]
└─$ rm -r studenti/matteo/amici

(kali㉿kali)-[~]
└─$ ls studenti/matteo

(kali㉿kali)-[~]
└─$
```

i) eliminare tutte le cartelle precedentemente create col comando **rm -r**. Io ho creato un backup dell'esercizio con **tar -czvf**

```
Session Actions Edit View Help
(kali㉿kali)-[~]
└─$ rm -r dos studenti windows

(kali㉿kali)-[~]
└─$ ls
backup_esercizio.tar.gz  Downloads  Projects  Templates
Desktop                 Music      Public    Videos
Documents               Pictures   'sudo inetsim'
```

Parte facoltativa:

Provare i comandi **w**, **who** e **whoami**

```
(kali@kali)-[~]
$ w
12:20:08 up 7:37, 1 user, load average: 0.44, 1.26, 1.71
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
kali - 04:43 0.00s 0.01s lightdm --se

(kali@kali)-[~]
$ who
kali seat0 2026-07-28 04:43 (:0)

(kali@kali)-[~]
$ who am i
kali

(kali@kali)-[~]
$ whoami
kali

(kali@kali)-[~]
```

1-2) aprire un terminale e leggere i manuali di job, ps e kill:

PS(1)
User Commands
PS(1)
NAME
ps - report a snapshot of the current processes.
SYNOPSIS
ps [options]
DESCRIPTION
ps displays information about a selection of the active processes. If you want a repetitive update of the selection and the displayed information, use top instead.
This version of ps accepts several kinds of options:
1 UNIX options, which may be grouped and must be preceded by a dash.
2 BSD options, which may be grouped and must not be used with a dash.
3 GNU long options, which are preceded by two dashes.
Options of different types may be freely mixed, but conflicts can appear. There are some synonymous options, which are functionally identical, due to the many standards and ps implementations that this ps is compatible with.
By default, ps selects all processes with the same effective user ID (euid=EUID) as the current user and associated with the same terminal as the invoker. It displays the process ID (pid=PID), the terminal as-
Manual page ps(1) line 1 (press h for help or q to quit)

KILL(1)
User Commands
KILL(1)
NAME
kill - send a signal to a process
SYNOPSIS
kill [options] <pid> [...]
DESCRIPTION
The default signal for kill is TERM. Use -l or -L to list available signals. Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0. Alternate signals may be specified in three ways: -9, -SIGKILL or -KILL. Negative PID values may be used to choose whole process groups; see the PGID column in ps command output. A PID of -1 is special; it indicates all processes except the kill process itself and init.
OPTIONS
<pid> [...]
Send signal to every <pid> listed.
-s <signal>
Specify the signal to be sent. The signal can be specified by using name or number. The behavior of signals is explained in signal(7) manual page.
Manual page kill(1) line 1 (press h for help or q to quit)

(kali@kali)-[~]
\$ bash
(kali@kali)-[~]
\$ help jobs
jobs: jobs [-lnprs] [jobspec ...] or jobs -x command [args]
Display status of jobs.
Lists the active jobs. JOBSPEC restricts output to that job. Without options, the status of all active jobs is displayed.
Options:
-l lists process IDs in addition to the normal information
-n lists only processes that have changed status since the last notification
-p lists process IDs only
-r restrict output to running jobs
-s restrict output to stopped jobs
If -x is supplied, COMMAND is run after all job specifications that appear in ARGS have been replaced with the process ID of that job's process group leader.
Exit Status:
Returns success unless an invalid option is given or an error occurs. If -x is used, returns the exit status of COMMAND.
(kali@kali)-[~]

3-4-5) lanciare vi pippo, aprire un nuovo terminale e vedere i processi (ps -ef) e cercare di killare il processo vi (kill PID di vi):

"pippo" [New] 0,0-1 All

root 81234 1 0 08:53 ? 00:00:00 /usr/lib/systemd/systemd
root 81235 1 0 08:53 ? 00:00:00 /usr/lib/systemd/systemd
root 81273 1 0 08:53 ? 00:00:00 /usr/lib/systemd/systemd
root 81292 2 0 08:53 ? 00:00:00 [psimon]
root 81311 2 0 08:53 ? 00:00:00 [psimon]
root 81433 2 0 12:17 ? 00:00:00 [kworker/u0:4-ext4-rsv-
root 81467 1 0 12:17 ? 00:00:00 /usr/lib/systemd/systemd
root 92558 2 0 12:37 ? 00:00:00 [kworker/1:2-events]
root 92558 2 0 12:42 ? 00:00:00 [kworker/0:0-mm_percpu_w
root 95807 2 0 12:43 ? 00:00:00 [kworker/1:1-ata_sff]
root 98716 81273 0 12:48 ? 00:00:00 system-userwork: waitin
root 98775 2 0 12:48 ? 00:00:00 [kworker/1:0-ata_sff]
root 98814 81273 0 12:48 ? 00:00:00 system-userwork: waitin
root 98815 81273 0 12:48 ? 00:00:00 system-userwork: waitin
kali 99820 1 0 12:50 ? 00:00:00 /usr/bin/qterminal
root 99827 99820 0 12:50 pts/0 00:00:00 /usr/bin/zsh
kali 99905 1 0 12:50 ? 00:00:00 /usr/bin/qterminal
kali 99912 99905 0 12:50 pts/1 00:00:00 /usr/bin/zsh
kali 99989 99827 0 12:50 pts/0 00:00:00 vi pippo
root 100207 2 0 12:50 ? 00:00:00 [kworker/0:1-events]
kali 100282 1281 0 12:51 ? 00:00:00 bwrap --unshare-all --di
kali 100283 100282 0 12:51 ? 00:00:00 bwrap --unshare-all --di
kali 100285 100283 0 12:51 ? 00:00:00 /usr/libexec/glycin-load
kali 100371 99912 0 12:51 pts/1 00:00:00 ps -ef
(kali@kali)-[~]

```
kali 100285 100283 0 12:51 ? 00:00:00 /usr/libexec/glycin-load
kali 100371 99912 0 12:51 pts/1 00:00:00 ps -ef

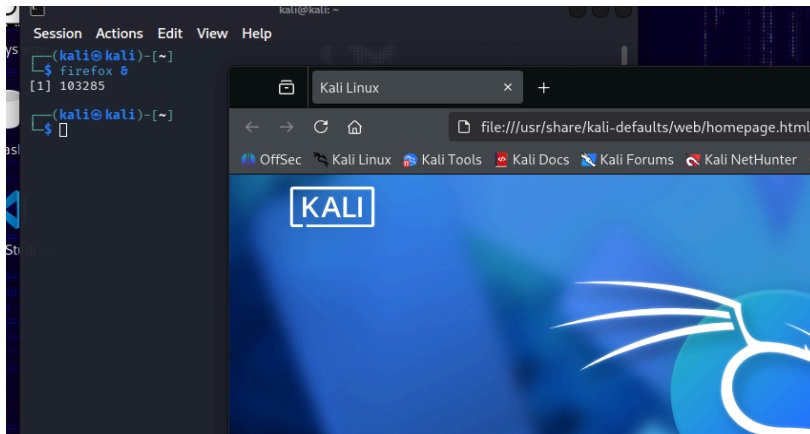
(kali@kali)-[~]
$ kill 99989

(kali@kali)-[~]
$
```

```
zsh: terminated vi pippo

(kali@kali)-[~]
$
```

6) lanciare firefox in background col comando **firefox &**



7) portarlo in background, dacchè lo abbiamo lanciato in background lo riportiamo in foreground per ripetere l'operazione, con i comandi **fg** e **bg**

```
(kali@kali)-[~]
$ firefox &
[1] 2180

(kali@kali)-[~]
$ jobs
[1] + running      firefox

(kali@kali)-[~]
$ fg
[1] + running      firefox
^Z
zsh: suspended    firefox

(kali@kali)-[~]
$ bg
[1] + continued    firefox

(kali@kali)-[~]
$
```

8) terminare il processo firefox quando in foreground

```
(kali@kali)-[~]
$ firefox &
[1] 12835

(kali@kali)-[~]
$ kill 12835

(kali@kali)-[~]
$
```

```
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.

[1] + terminated    firefox
```

9) verificare lo spazio occupato su disco col comando **df -h**, per una cartella

specifica `du -sh [nome cartella]`

```
(kali㉿kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            889M   0    889M   0% /dev
tmpfs           197M  964K   196M   1% /run
/dev/sda1       79G   18G   57G   24% /
tmpfs           985M  4.0K   985M   1% /dev/shm
none            1.0M   0    1.0M   0% /run/credentials/systemd-journald.serv
ice
tmpfs           985M  8.0K   985M   1% /tmp
none            1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M  108K   197M   1% /run/user/1000

(kali㉿kali)-[~]
$
```